



Basi di Dati
Progetto A.A. 2024/2025

“Sistema informativo Scuola di lingua Inglese”

0240885
Andrea Tamburrini

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti	3
3. Progettazione concettuale.....	6
4. Progettazione logica	9
5. Progettazione fisica	14

1. Descrizione del Minimondo

1 Si progetti un sistema informativo per la gestione dei corsi di lingua inglese, tenuti
2 presso un istituto di insegnamento. I corsi sono organizzati per livelli. Ciascun livello
3 è identificato dal nome del livello stesso (ad esempio Elementary, Intermediate, First
4 Certificate, Advanced, Proficiency); inoltre è specificato il nome del libro di testo e
5 se viene richiesto di sostenere un esame finale.

6 I corsi sono identificati univocamente dal nome del livello cui afferiscono e da un
7 codice progressivo, necessario per distinguere corsi che fanno riferimento allo stesso
8 livello. Per ciascun corso sono note la data di attivazione, il numero e le informazioni
9 anagrafiche degli iscritti e l'elenco dei giorni ed orari in cui è tenuto.

1 Per gli insegnanti sono noti il nome, l'indirizzo, la nazione di provenienza, ed i corsi
2 a cui sono stati assegnati. Ad un corso può essere assegnato più di un insegnante,
3 assicurandosi che in una determinata fascia oraria un insegnante sia assegnato ad un
4 solo corso.

5 Per gli allievi sono noti il nome, un recapito, il corso a cui sono iscritti, la data di
6 iscrizione al corso e il numero di assenze fatte finora (è di interesse tenere traccia dei
7 giorni specifici in cui un allievo è stato assente).

8 Il personale amministrativo della scuola deve poter inserire all'interno del sistema
9 informativo tutte le informazioni legate ai corsi ed agli insegnanti e possono generare
dei report indicanti, su base mensile, quali attività hanno svolto gli insegnanti. Il
personale di segreteria gestisce le iscrizioni degli utenti della scuola ai corsi. Gli
insegnanti possono generare dei report indicanti la propria agenda, su base
settimanale.

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
14	Allievi	Studente	Termine più consono per una scuola
20	Utenti	Studenti	La segreteria gestisce l'iscrizione degli studenti
9	Iscritti	Studenti iscritti	Gli iscritti si riferisce ovviamente agli studenti dello specifico corso
9	Giorni	Giorni della settimana	È più specifico rispetto a giorni
9	Orari	Fasce orarie	Più chiaro ed identifica meglio la durata delle lezioni di uno specifico corso
14	Recapito	Telefono	Meno generico rispetto a recapito

Specificazione disambiguata

Si progetti un sistema informativo per la gestione dei corsi di lingua inglese, tenuti presso un istituto di insegnamento. I corsi si riferiscono ad uno specifico livello di preparazione. Ciascun livello è identificato dal nome del livello stesso (ad esempio Elementary, Intermediate, First Certificate, Advanced, Proficiency); inoltre per esso è specificato il nome del libro di testo e l'eventuale esame finale.

I corsi sono identificati univocamente dal nome del livello cui afferiscono e da un codice progressivo, necessario per distinguere corsi che fanno riferimento allo stesso livello. Per ciascun corso sono note la data di inizio, il numero e le informazioni anagrafiche degli studenti iscritti e l'elenco dei giorni della settimana e le fasce orarie in cui è tenuto.

Per gli insegnanti sono noti il nome, l'indirizzo, la nazione di provenienza, ed i corsi a cui sono stati assegnati. Un corso può essere tenuto da più insegnanti, ma ciascun insegnante non può essere assegnato a più corsi nella stessa fascia oraria.

Per gli Studenti sono noti il nome, il numero di telefono, il corso a cui sono iscritti, la data di iscrizione al corso e il numero di assenze (è di interesse tenere traccia dei giorni della settimana in cui un allievo è stato assente).

Il personale amministrativo della scuola deve poter inserire all'interno del sistema informativo tutte le informazioni legate ai corsi ed agli insegnanti. Inoltre, deve poter generare report mensili che indicano, per ogni insegnante, i corsi che ha tenuto. Il personale di segreteria gestisce le iscrizioni degli studenti della scuola ai rispettivi corsi. Gli insegnanti possono generare dei report indicanti l'elenco delle lezioni da tenere con le rispettive fasce orarie, su base settimanale.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Corsi	Rappresenta serie di lezioni riguardante uno specifico livello di preparazione nella lingua inglese	Percorso di studi	Livello Insegnante Studente
Studente	Entità del sistema che partecipa ai corsi	Utenti della scuola	Corsi Iscrizione Lezione
Insegnante	Colui designato per impartire lezioni nei rispettivi corsi	Docente	Corsi Lezione Fascia oraria
Livello	Stadio di preparazione nella lingua inglese	Grado Competenza	Corsi
Iscrizione	Il personale di segreteria deve gestire le iscrizioni degli studenti ai rispettivi corsi	Registrazione	Studente Corsi
Lezione	Sessione didattica che fa parte di un Corso, in cui gli studenti partecipano.	Insegnamento	Insegnante Corso Studente

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a Insegnante

Per gli insegnanti sono noti il nome, l'indirizzo, la nazione di provenienza, ed i corsi a cui sono stati assegnati. Un corso può essere tenuto da più insegnanti, ma ciascun insegnante non può essere assegnato a più corsi nella stessa fascia oraria.

Gli insegnanti possono generare dei report indicanti l'elenco delle lezioni da tenere con le rispettive fasce orarie, su base settimanale.

Frasi relative a Studenti

Per gli Studenti sono noti il nome, il numero di telefono, il corso a cui sono iscritti, la data di iscrizione al corso e il numero totale di assenze (è di interesse tenere traccia del giorno della settimana in cui uno studente è stato assente).

Frasi relative a Corsi

I corsi si riferiscono ad uno specifico livello di preparazione.

I corsi sono identificati univocamente dal nome del livello cui afferiscono e da un codice progressivo, necessario per distinguere corsi che fanno riferimento allo stesso livello. Per ciascun corso sono note la data di inizio, il numero e le informazioni anagrafiche degli studenti iscritti e l'elenco dei giorni della settimana e le fasce orarie in cui è tenuto.

Frasi relative a Livello

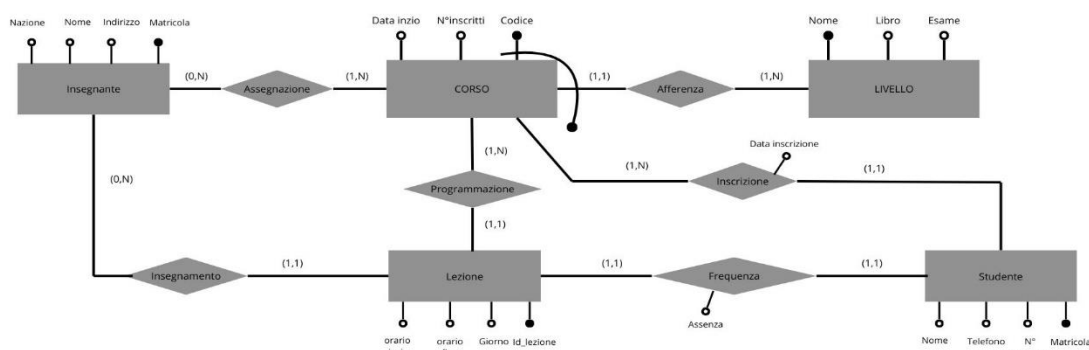
Ciascun livello è identificato dal nome del livello stesso (ad esempio Elementary, Intermediate, First Certificate, Advanced, Proficiency); inoltre per esso è specificato il nome del libro di testo e l'eventuale esame finale.

Frasi relative a Iscrizione

Il personale di segreteria gestisce le iscrizioni degli studenti della scuola ai rispettivi corsi. Per ciascun corso sono note la [...] il numero e le informazioni anagrafiche degli studenti iscritti.

3. Progettazione concettuale

Costruzione dello schema E-R



L'intento principale nella realizzazione dello schema E-R è stato quello di cercare di definire entità e associazioni atomiche una per ogni singolo elemento del dominio del problema in modo già da rendere chiaro e ben visibile le varie componenti/attori del sistema informativo.

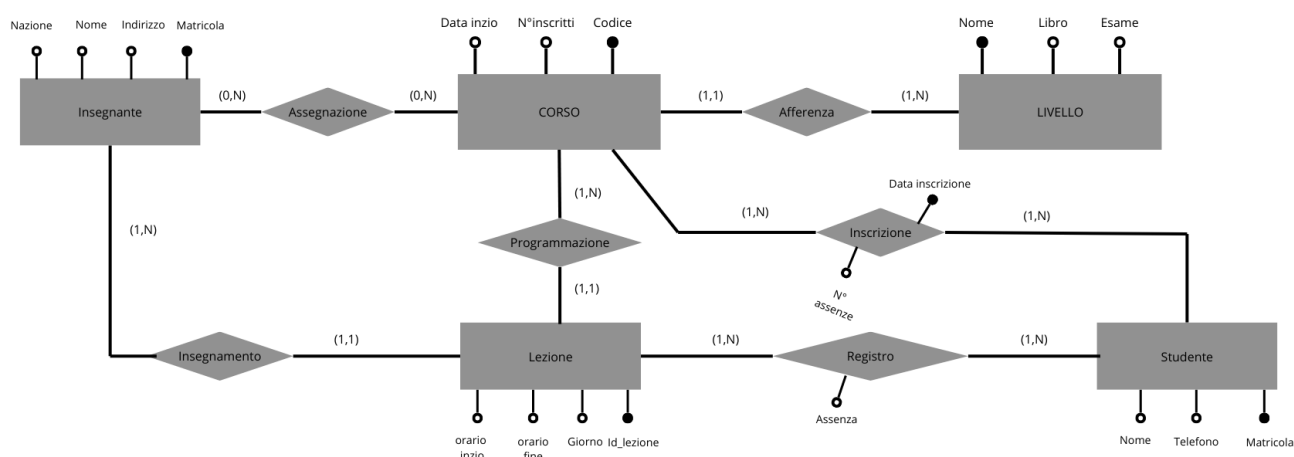
In questa prima versione dello schema come si può notare non era funzionale e in particolare non rispettava al meglio le specifiche della traccia come ad esempio nell'associazione Frequenza le cardinalità (1;1) per entrambe le entità sono errate, il suo scopo principalmente era quello di tenere traccia della frequenza di ogni singolo studente alle rispettive lezioni in modo da recuperare in particolare le informazioni di interesse come la data di assenza, ma così com'è, permetteva l'inserimento di un dato studente una e una sola volta nella relazione, non tenendo traccia di ciò di cui avevamo bisogno.

Un'altra problematica era la presenza dell'attributo n°_assenze nell'entità Studente, il quale non rendeva per niente pratico il conteggio e la lettura delle rispettive assenze.

Altro problema è l'associazione Inscrizione dove anche qui le cardinalità non permettevano di tenere traccia delle informazioni di interesse e in particolare non permettevano così ad uno stesso studente di iscriversi a più corsi durante l'anno accademico.

Altro problema è l'identificazione esterna del corso con l'entità livello, problema che sorgeva nel momento dell'assorbimento delle varie associazioni, le quali ereditavano l'identificazione esterna rendendo più complessa la definizione dell'entità Corso.

Integrazione finale



Questa è la versione finale dello schema E-R, in particolare è stata eliminata l'identificazione esterna dell'entità Corso con la relazione afferenza in quanto il corso può essere identificato tramite il codice univoco.

Nella relazione Inscrizione è stata generata una chiave composta che sfrutta la traduzione delle associazioni in relazioni sfruttando l'ereditarietà delle chiavi dell'entità corso(codice) e studente(matricola) insieme a Data Inscrizione ,così permette di avere più iscrizioni dello stesso studente a più corsi.

In più è stato aggiunto l'attributo n° assenze, associato in questo modo ad uno specifico studente e il rispettivo corso.

In questa fase dato anche lo schema precedente non si sono verificati conflitti sui nomi utilizzati.

Regole aziendali

- Uno studente non può iscriversi a più di un corso contemporaneamente in funzione della data di iscrizione
- Ad un Corso può essere assegnato più di un insegnante limitato a due insegnanti per uno specifico corso.
- Il giorno e la fascia oraria di una specifica lezione assegnata ad un insegnante non può coincidere con l'assegnazione ad un'altra lezione
- Il numero massimo di assenze per un dato studente è stato imposto pari a 10.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Livello	Grado di insegnamento della lingua inglese	Nome,Libro,Esame	Nome
Corso	Ciclo di lezioni di uno specifico livello della lingua inglese	Codice,n°iscritti,Data attivazione	Codice,Afferenza
Studente	Le Informazioni su uno studente che si iscrive	Nome,Cognome,n°telefono,Matricola	Matricola
Insegnante	Le informazioni su un insegnante che fa parte della scuola	Nome,Indirizzo,Nazione,Matricola	Matricola
Lezione	Informazioni di una specifica lezione riguardante uno specifico corso	Orario inizio,Orario fine,giorno settimana,Id_lezione	Id_lezione

4. Progettazione logica

Volume dei dati

Assunzioni preliminari:

- Numero di studenti per una scuola di medie dimensioni in un anno pari a 5000
- Ogni studente si iscrive in media a due corsi nell'anno accademico (es. B1 e B2)
- Ogni corso ha 4 lezioni a settimana per una durata di 12 settimane totale 48 lezioni totali.
- Per ogni corso in media sono assegnati due insegnanti che si dividono equamente le lezioni previste per il corso
- Supponiamo che abbiamo 3 cicli di lezioni(slot) all'anno es: Gennaio-Marzo, Aprile-Giugno, Settembre-Novembre.
- Supponiamo che la frequenza di partecipazione ai corsi siano le seguenti percentuali e che ogni classe è composta da 20 studenti:
- A1=10%, A2=20%, B1 e B2=25% C1 e C2=10% ottenendo un numero di corsi annui pari a 251.
- Supponiamo per semplicità che gli studenti ossia 5000 si dividano equamente nei tre slot disponibili di lezioni avendo così circa 1667 studenti per slot
- Supponiamo anche che gli insegnanti vengano assunti ad inizio anno accademico e che tengano sempre gli stessi corsi.
- Supponiamo anche che ogni insegnante possa tenere per slot di lezioni al massimo due corsi dividendo equamente il numero di lezioni totali ossia 36 con l'altro insegnante, in definitiva è come se seguisse un unico corso per slot.

Concetto nello schema	Tipo ¹	Volume atteso
Studente	E	5.000
Inscrizione	R	10.000
Corso	E	249
Lezione	E	11.952
Insegnante	E	83
Insegnamento	R	11.952
Afferenza	R	249
Assegnazione	R	498
Programmazione	R	8.946
Registro	R	480.000
Livello	E	6

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
Op1	Creazione nuovo corso	83/trimestrale

¹ Indicare con E le entità, con R le relazioni

Op2	Inserimento nuovo insegnante	83/annuale
Op3	Assegnazione insegnante ad un corso	166/trimestrale
Op4	Report attività insegnanti	5/mensile
Op5	Inscrizione nuovo studente	1667/trimestrale
Op6	Inscrizione studente ad uno specifico corso	3.333/trimestrale
Op7	Report lezioni di un singolo insegnante	83/settimanale
Op8	Appello	940/giorno
Op9	Login	200/giorno

Costo delle operazioni

Op1-Creazione nuovo corso:

Supponendo che la creazione di nuovi corsi avvenga all'inizio di ogni ciclo di lezioni a cadenza trimestrale escludendo luglio agosto e dicembre abbiamo 3 cicli di lezioni trimestrali da cui:

Accesso alla sola entità corso in scrittura con costo due volte lettura da cui:

Costo per trimestre $2 \cdot 83 = 166$ operazioni/trimestre e 498 operazioni/anno

Op2-Inserimento nuovo professore:

Calcolando che i professori vengano assunti all'inizio dell'anno accademico e che non ci sia un ricambio dei professori consideriamo una media di 83 professori all'anno.

Da cui ci sarà un accesso in scrittura ottenendo:

$$2 \cdot 83 = 166 \text{ operazioni/anno}$$

Op3-Assegnazione professore ad un corso:

Un accesso in lettura all'entità professore, un accesso in lettura all'entità corso e una successiva scrittura nella relazione assegnazione da cui :

$$4 \cdot 83 \cdot 2 = 664 \text{ op/trimestre ,}$$

$83 \cdot 2$ perchè consideriamo che un insegnante sia assegnato a due corsi.

Op4-Report attività insegnante a base mensile:

Dato che un corso ha 4 lezioni a settimana e un prof si divide equamente le lezioni di uno specifico corso con il professore associato in tutto eseguirà comunque 4 lezioni a settimana da cui $4 \cdot 4 = 16$ lezioni al mese.

Quindi:

1 accesso in lettura all'entità professore

16 accessi in lettura alla relazione insegnamento

16 accessi all'entità lezione per recuperare le info della lezione specifica

Totale 33 accessi in lettura

In media abbiamo 83 professori

Totale operazioni: $33 \cdot 83 = 2739$ op/mensile.

(L'accorpamento della relazione insegnamento con lezione elimina i 16 accessi) da cui:

totale: $17 \text{ accessi in lettura} \cdot 83 = 1411$ mensile

Op5-Immatricolazione nuovo studente:

Essendo nuovo si esegue un solo accesso in scrittura all'entità studente da cui 2 op per studente.
In media considerando le iscrizioni valide solo all'inizio di ogni ciclo di lezioni trimestrali e che in media abbiamo 1667 studenti per trimestre
Totale operazioni: $2 \cdot 1667 = 3334$ op/trimestre

Op6-Inscrizione studente specifico corso:

Assumendo che in media uno studente nel corso dell'anno può iscriversi a due corsi esempio uno base e uno avanzato usiamo il dato 3.333 iscrizioni corso/trimestre.
Un accesso in lettura a studente, un accesso in lettura all'entità corso e un accesso in scrittura all'entità iscrizione per singolo studente totale op=4 totale op: $3.333 \cdot 4 = 13.332$ op/trimestre.

Op7-Report agenda professore su base settimanale:

1 accesso in lettura a professore 4 accessi in lettura a insegnamento e 4 accessi in lettura all'entità lezione per le sue info specifiche totale 9 op singolo professore totale op: $9 \cdot 83 = 747$ op/settimanale

Op8-Registrazione assenza:

Viene eseguita dal prof che tiene la lezione da cui e dal carattere molto aleatorio:
Un accesso in lettura a lezione, un accesso in lettura all'entità studente, una scrittura in registro e un accesso in lettura e scrittura per aggiornare l'attributo "numero assenze", da cui 7 operazioni/singolo studente.
 948 al giorno perché sono 83 corsi da 4 lezioni a settimana da cui $(83 \cdot 4) / 7 = 47$ circa
Sono 20 studenti per classe da cui $47 \cdot 20 = 940$ record al giorno.
Da cui $7 \cdot 940 = 6.636$ op/giorno.

Op9-login:

Viene eseguita una sola lettura della tabella login da cui:
totale: 200 op/giorno.

Ristrutturazione dello schema E-R

Dal punto di vista delle ridondanze dopo un'analisi fatta sul costo delle operazioni ho scelto di mantenere gli attributi `n°_iscritti` e `n°_assenze` nelle varie entità Corso e Iscrizione in quanto l'aggiornamento di tali attributi viene effettuato di volta in volta con una semplice scrittura anche se di costo doppio rispetto ad una operazione in lettura.

Se invece non presenti all'atto della richiesta di tale informazioni il sistema avrebbe dovuto fare una scansione delle varie relazioni per fare tale conteggio e al crescere delle dimensioni del sistema avrebbe portato ad un rallentamento di tale operazioni, invece così facendo con un semplice accesso in lettura vengono recuperati tali dati.

Come identificatori primari sono stati scelti per le rispettive relazioni:

Codice per la relazione corso, Nome per la relazione Livello, Matricola per l'entità insegnante, Matricola per l'entità studente, Id_lezione per la relazione Lezione generato dal sistema per discriminare le varie lezioni così da avere una chiave primaria di un solo attributo, Insegnante e corso per la relazione Assegnazione ed infine Corso(codice),Data Iscrizione, Studente(matricola) per la relazione Iscrizione.

Trasformazione di attributi e identificatori

In questa fase è stata adottata una trasformazione per quanto riguarda la relazione Iscrizione.

Come da definizione è stata realizzata una relazione Iscrizione con i propri attributi e come identificatori una chiave composta dalla Data Iscrizione unita al codice corso e matricola studente.

Traduzione di entità e associazioni

Livello(Nome, Libro, Esame);

Corso(Codice, Livello, Data_inizio, n°_iscritti);

Studente(Matricola, Cognome, Nome, Telefono);

Insegnante(Matricola, Nome, Nazione, Indirizzo);

Registro(Studente, Lezione, Assenza);

Lezione(ID_lezione, Corso, Insegnante, orario_inizio, orario_fine, data.);

Iscrizione(Corso, Studente, Data_inscrizione, n°assenze);

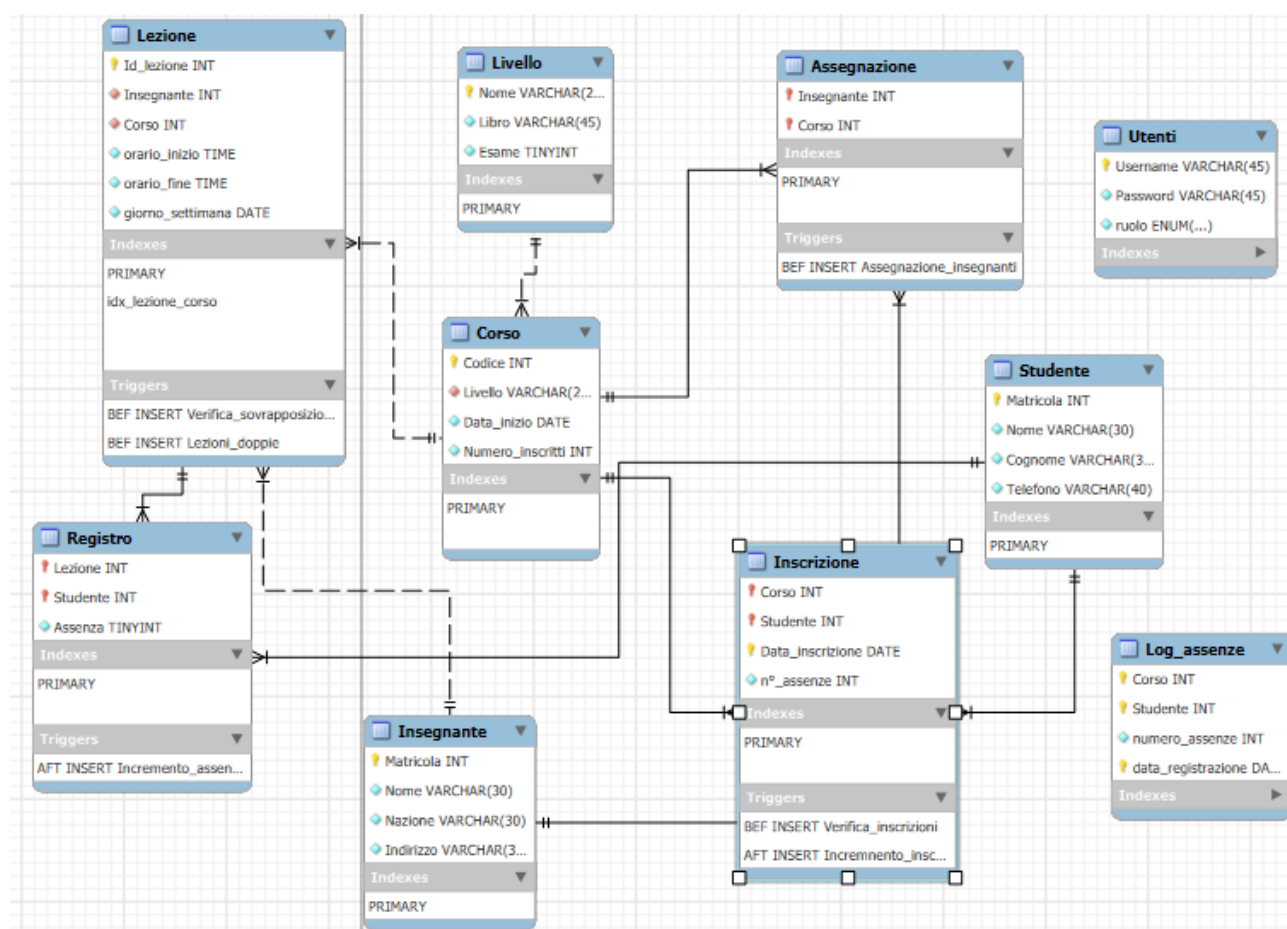
Assegnazione(Insegnante, Corso);

E' stata effettuata una ristrutturazione dal punto di vista delle associazioni:

L'associazione Insegnamento è stata assorbita dall'entità Lezione.

L'associazione Programmazione è stata assorbita dall'entità Lezione.

L'associazione Afferenza è stata assorbita dall'entità Corso.



Normalizzazione del modello relazionale

Lo schema già dalle prime fasi di sviluppo si è cercato di non creare insiemi di informazioni di tipo eterogeneo per evitare il comparire di dipendenze funzionali non banali ossia dettate dalle chiavi stesse.

Da analisi effettuate risulta essere già in BNFC in quanto tutte le dipendenze funzionali nelle varie relazioni di tipo $X \rightarrow A_i$, all'interno dell'insieme di attributi X è sempre presente una chiave per la singola relazione.

Da cui ogni valore degli attributi dipende dalla chiave in X per cui rispettiamo la condizione di Boyce e Codd.

5. Progettazione fisica

Utenti e privilege

Sono stati identificati 3 tipi di utente:

Amministratore;

Insegnante;

Segreteria;

Ovviamente è stata inserita una tabella Utenti per gestire l'autenticazione al sistema e il recupero dei rispettivi ruoli(privilegi).

Privilegi Amministratore:

Sono assegnati i privilegi di Select, Insert, Delete, Update sulla tabella Corso.

Sono assegnati i privilegi di Select, Insert, Delete, Update sulla tabella Insegnanti.

Sono assegnati i privilegi di Select, Insert, Update sulla tabella Lezioni per estrapolare il report mensile sulle attività degli insegnanti e per definire le lezioni per i vari corsi e i rispettivi insegnanti per le varie lezioni.

Sono assegnati i privilegi di Select, Insert, Delete, Update sulla tabella Assegnazioni per gestire l'assegnazione degli insegnanti ai corsi.

Privilegi Insegnante:

Sono previsti i privilegi di Select, Insert, Update, sulla tabella Registro e Iscrizione per annotare le presenze/assenze degli studenti per le sue relative lezioni e reperirne il numero complessivo di assenze.

Sono previsti i privilegi di Select sulla tabella lezioni per generare il proprio report settimanale della relativa agenda.

Privilegi Segreteria:

Sono previsti i privilegi di Select, Insert, Delete e update sulla tabella Studenti per l'operazione di immatricolazione.

Sono previsti i privilegi di Select, Insert, Delete e update sulla tabella Iscrizioni per la gestione dell'iscrizione degli studenti ai rispettivi corsi.

Strutture di memorizzazione

Tabella Livello		
Colonna	Tipo di dato	Attributi ²
Nome	Varchar(20)	PK,NN
Libro	Varchar(45)	NN
Esame	TINYINT	NN

Tabella Corso		
Colonna	Tipo di dato	Attributi ³
Codice	INT	PK,AI,NN,UQ
Livello	varchar(20)	FK,NN
Data_inizio	DATE	NN
n°_iscritti	INT	NN,UN,DEFAULT=0
		UNIQUE(Codice,Livello)

Tabella Insegnante		
Colonna	Tipo di dato	Attributi ⁴
Matricola	INT	PK,AI
Nome	varchar(30)	NN
Nazione	varchar(30)	NN
Indirizzo	varchar(30)	NN

Tabella Studente		
Colonna	Tipo di dato	Attributi ⁵
Matricola	INT	PK,AI
Nome	varchar(30)	NN
Cognome	varchar(30)	NN
Telefono	varchar(40)	NN

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella Registro		
Colonna	Tipo di dato	Attributi ⁶
Lezione	INT	PK,FK,NN
Studente	INT	PK,FK,NN
Assenza	TINYINT	NN

Tabella Iscrizione		
Colonna	Tipo di dato	Attributi ⁷
Corso	INT	PK,FK
Studente	INT	PK,FK
Data_inscrizione	DATE	PK
n° assenze	INT	NN,UN,DEFAULT=0

Tabella Lezione		
Colonna	Tipo di dato	Attributi ⁸
Id_lezione	INT	PK,AI
Insegnante	INT	FK,NN
Corso	INT	FK,NN
orario_inizio	TIME	NN
orario_fine	TIME	NN
giorno_settimana	DATE	NN

Tabella Assegnazione		
Colonna	Tipo di dato	Attributi ⁹
Insegnante	INT	PK,FK,NN
Corso	INT	PK,FK,NN

PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella Utenti		
Colonna	Tipo di dato	Attributi ¹⁰
username	Varchar(45)	PK,NN
password	Char(32)	NN
Ruolo	ENUM('amministratore,segreteria,insegnante')	NN

Tabella Utenti		
Colonna	Tipo di dato	Attributi ¹¹
Corso	INT	PK,NN
Studente	INT	PK,NN
Data_registrazione	DATE	PK,NN
N°_assenze	INT	NN

Indici

Tabella <Lezione>	
Indice <idx_lezione_corso>	Tipo ¹² :
Corso	IDX

Tabella <Lezione>	
Indice <idx_lezione_insegnante>	Tipo ¹³ :
Insegnante	IDX

Per la tabella Lezione, ho creato due indici separati su Corso e Insegnante perché queste colonne sono utilizzate intensivamente in operazioni distinte. L'indice su Corso accelera il recupero di tutte le lezioni associate a un determinato corso mentre il secondo indice secondario ottimizza le query relative all'agenda dei docenti, permettendo un rapido accesso a tutte le lezioni di un insegnante specifico

¹² IDX = index, UQ = unique, FT = full text, PR = primary.

¹³ IDX = index, UQ = unique, FT = full text, PR = primary.

Tabella <Corso>	
Indice <idx_corso_livello>	Tipo ¹⁴ :
Livello	IDX

Questo indice secondario è stato implementato perché il ‘livello’ è utilizzato spesso sia quando si filtrano i corsi disponibili che durante le operazioni di iscrizione degli studenti.

Tabella <Inscrizione >	
Indice <idx_inscrizione_studente_corso >	Tipo ¹⁵ :
Studente,Corso	IDX

Questo indice secondario composto è stato inserito perché queste due colonne vengono sempre utilizzate insieme nelle query anche per esempio per incrementare il numero di assenze eseguito dal trigger apposito per uno specifico studente per uno specifico corso

Tabella <Assegnazione >	
Indice <idx_assegnazione_insegnante >	Tipo ¹⁶ :
Insegnante	IDX

Tabella <Assegnazione >	
Indice <idx_assegnazione_corso>	Tipo ¹⁷ :
Corso	IDX

Questi due indici secondary sono stati inseriti separatamente per velocizzare le query che utilizzano questi elementi in modo separato, alcune per reperire i corsi assegnati ad uno specifico insegnante in particolare anche per supportare il lavoro del trigger che verifica il Massimo numero di assegnazioni e dall'altra parte le query che invece hanno bisogno di recuperare gli insegnanti assegnati ad uno specifico corso.

¹⁴ IDX = index, UQ = unique, FT = full text, PR = primary.

¹⁵ IDX = index, UQ = unique, FT = full text, PR = primary.

¹⁶ IDX = index, UQ = unique, FT = full text, PR = primary.

¹⁷ IDX = index, UQ = unique, FT = full text, PR = primary.

Trigger

Aggiornamento numero assenze:

```
CREATE TRIGGER `Incremento_assenze`
AFTER INSERT ON `scuola_lingue`.`Registro`
FOR EACH ROW
BEGIN
    DECLARE `Codice_corso` int;
    IF new.Assenza=1 THEN
        SELECT `codice` INTO `Codice_corso`
        FROM `scuola_lingue`.`Registro` as R join `scuola_lingue`.`Lezione` as L ON
        R.`Lezione`=L.`ID_lezione` Join `scuola_lingue`.`Corso` as C ON L.`Corso`=C.`Codice`
        LIMIT 1;
        UPDATE `scuola_lingue`.`Inscrizione`
        SET `n°_assenze`=`n°_assenze`+1
        WHERE `Studente`=NEW.`Studente` and `Corso`= `Codice_corso`;
        IF (SELECT `n°_assenze` FROM `scuola_lingue`.`Inscrizione` WHERE `Studente`=
        NEW.`Studente`) > 10 THEN
            SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT='Numero massimo di assenze raggiunto';
        END IF;
    END IF;
END;
```

Verifica assegnazione insegnanti:

```
CREATE TRIGGER `Assegnazione_insegnanti`
BEFORE INSERT ON `scuola_lingue`.`Assegnazione`
FOR EACH ROW
BEGIN
    DECLARE `numero_corsi` INT;

    SELECT COUNT(*) INTO `numero_corsi`
    FROM `scuola_lingue`.`Assegnazione`
    WHERE `Insegnante`=new.`Insegnante`;
    IF `numero_corsi` >= 2 THEN
        SIGNAL SQLSTATE '45001'
        SET MESSAGE_TEXT='Numero massimo corsi raggiunto per il seguente insegnante';
    END IF;
END;
```

Verifica Sovrapposizione fasce orarie Insegnanti:

```
CREATE TRIGGER `Verifica_sovrapposizione`
BEFORE INSERT ON `scuola_lingue`.`Lezione`
FOR EACH ROW
BEGIN
```

```

IF EXISTS (SELECT 1 FROM LEZIONE WHERE `Corso`=new.`Corso`
and `Insegnante`=new.`Insegnante`
and `giorno_settimana`=new.`giorno_settimana`
and (new.`orario_inizio` between `orario_inizio` and `orario_fine`
or new.`orario_fine` between `orario_inizio` and `orario_fine`
or (new.`orario_inizio`<=`orario_inizio` and new.`orario_fine`>=`orario_fine`))) THEN
SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='IL professore specifico ha già una lezione per la specifica fascia oraria';
END IF;
END;

```

Aggiornamento numero iscritti:

```

CREATE TRIGGER `Incremento_inscrizioni`
AFTER INSERT ON `scuola_lingue`.`Inscrizione`
FOR EACH ROW
BEGIN
UPDATE `scuola_lingue`.`Corso`
SET `Corso`.`Numero_iscritti`=`Numero_iscritti`+1
WHERE `Codice`=NEW.`Corso`;
END;

```

Verifica numero iscritti massimo:

```

CREATE TRIGGER `Verifica_inscrizioni`
BEFORE INSERT ON `scuola_lingue`.`Inscrizione`
FOR EACH ROW
BEGIN
DECLARE num_iscritti int;
SELECT COUNT(*) into num_iscritti
FROM `scuola_lingue`.`Inscrizione`
WHERE `Corso`=new.`Corso`;
IF num_iscritti >=20 THEN
SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='numero massimo iscritti raggiunto per lo specifico corso';
END IF;
END;

```

Verifica inserimento lezione multiplo:

```

CREATE TRIGGER `Lezioni_doppie`
BEFORE INSERT ON `scuola_lingue`.`Lezione`
FOR EACH ROW
BEGIN
IF EXISTS(SELECT 1 FROM `scuola_lingue`.`Lezione` WHERE `Corso`=new.`Corso` and
`giorno_settimana`=new.`giorno_settimana` and `orario_inizio`=new.`orario_inizio`) THEN
SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='è già presente una lezione per lo specifico corso e la specifica data e
ora';
END IF;

```

END;

Eventi

Questo **evento** è previsto staticamente ogni 5 anni per permettere di eliminare le iscrizioni obsolete e viene istanziato in fase di configurazione.

Eliminazione iscrizioni obsolete:

```
CREATE EVENT PuliziaIscrizioniObsolete
ON SCHEDULE EVERY 5 YEAR
DO
BEGIN
DELETE      FROM      `scuola_lingue`.`Iscrizione`      WHERE      `Data_inscrizione`      <
DATE_SUB(CURDATE(), INTERVAL 5 YEAR);
END;
```

```
CREATE EVENT `Verifica_assenze_mensili`
ON SCHEDULE EVERY 1 MONTH
DO
BEGIN
INSERT                                           INTO
`scuola_lingue`.`Log_assenze`(`Corso`,`Studiante`,`data_registrazione`,`numero_assenze`)
SELECT `Corso`,`Studiante`,CURDATE(),`n°_assenze`
FROM `scuola_lingue`.`Iscrizione`
WHERE `n°_assenze` >=10;
END;
```

Questo evento si attiva ogni mese facendo una scansione di tutti gli studenti utilizzando la tabella iscrizione e andando a riportare le generalita come corso,matricola studente e e numero di assenze con la data di registrazione all'interno di una tabella di log detta appunto Log_assenze.

Aggiornamento assenze:

Nel momento in cui un insegnante inserisce nella tabella registro l'assenza di uno specifico studente identificato dal numero di matricola se il valore inserito è True, il trigger si attiva ed aggiorna il contatore del n°_assenze nella tabella iscrizioni, ovviamente ci sono anche dei controlli nel trigger per rispettare il non superamento di un numero di assenze superiore a 20.

Verifica assegnazione Insegnati:

Questo trigger viene attivato nel momento in cui la procedura assegnazione_insegnante viene invocata infatti esso è di tipo before e verifica se per quel dato insegnante con questo nuovo inserimento violerà o meno il limite di due corsi assegnati per esso.

Verifica Sovrapposizione fasce orarie Insegnanti:

L'evento che attiva questo trigger corrisponde con le attività della procedura nuova lezione nel momento in cui ad un dato insegnante per una specifica fascia oraria è già stata assegnata in precedenza una lezione e si tenta di effettuare un nuovo inserimento.

Aggiornamento numero iscritti:

L'evento che attiva questo trigger corrisponde con le attività della procedura nuova_iscrizione, il quale dopo l'inserimento eseguito va ad aggiornare il counter n°_iscritti della tupla corrisponde al corso specificato.

In particolare in funzione di questo evento viene attivato prima dell'inserimento fatto dalla procedura un altro trigger che va a verificare il raggiungimento del numero massimo di iscritti il trigger di nome verifica_iscritti essendo infatti di tipo BEFORE.

Verifica inserimento lezione multiplo:

L'evento previsto che attiva questo trigger è l'operazione di insert effettuata dalla procedura nuova_lezione nel momento in cui si tenta di assegnare ad un insegnante più di una lezione in una specifica fascia oraria per il quale è già assegnata una precedente lezione.

Viste

```
CREATE VIEW `Report` as  
  SELECT `Nome`,`Matricola`,`Codice`,`Livello`,COUNT(*) as N°_lezioni
```

```

FROM `scuola_lingue`.`Insegnante` I JOIN `scuola_lingue`.`Assegnazione` A JOIN
`scuola_lingue`.`Corso` C JOIN `scuola_lingue`.`Lezione` L
WHERE I.Matricola=A.Insegnante AND A.Corso = C.Codice AND C.Codice = L.Corso
GROUP BY `Nome`,`Matricola`,`Codice`,`Livello`;

```

Questa vista viene utilizzata nella procedura agenda.

Essa viene invocata esclusivamente dalla tipologia utente professore.

I risultati della seguente view poi vengono filtrati dalla procedura suddetta recuperando la data esatta della settimana corrente tramite la funzione del dbms curdate() e dateofweek();

CREATE

VIEW`Report\_professori`(Matricola,ID_corso,Nome_corso,Lezione,inizio,fine,giorno_settimana)

ASSELECT

`Matricola`,`Codice`,`Livello`,`ID\_lezione`,`orario\_inizio`,`orario\_fine`,`giorno\_settimana`

```

FROM `scuola_lingue`.`Insegnante` I JOIN `scuola_lingue`.`Lezione` L ON
I.Matricola=L.Insegnante JOIN Corso C on L.Corso=C.Codice
order by `Matricola`;

```

Questa vista viene utilizzata prevalentemente dalla procedura report mensile utilizzata esclusivamente dal personale amministrativo dell'istituto in quanto restituisce tutte le attività di tutti i professori filtrando i risultati della seguente view tramite il recupero del mese corrente fatto dal dbms tramite la funzione curdate nella suddetta procedura.

Stored Procedures e transazioni

Per tutte le procedure sono state adottate transazioni ed il fatto risiede nella possibilità di sfruttare le loro proprietà acide che vengono garantite dal DBMS, non solo per il corretto stato della base di dati dopo le modifiche effettuate ma anche per gestire la concorrenzialità delle varie procedure che possono essere invocate in parallelo da più utenti del sistema ad esempio, più di un utente della segreteria potrebbe manipolare i dati degli studenti o anche più amministratori dell'istituto potrebbero manipolare contemporaneamente il contenuto della tabella Lezione o le informazioni riguardante gli insegnanti.

Creazione nuovo Corso:

CREATE PROCEDURE `Crea\_corso`(in Livello_corso VARCHAR(20),in Data_inizio DATE)

BEGIN

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

```

ROLLBACK;
RESIGNAL;
END;
set transaction isolation level repeatable read;
START TRANSACTION;
IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Livello` WHERE
`scuola_lingue`.`Livello`.`Nome`=Livello_corso)
    THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Il livello inserito non
corrisponde con quelli esistenti';
END IF;
INSERT INTO `scuola_lingue`.`Corso`(`Livello`,`Data_inizio`,`Numero_inscritti`) values
(Livello_corso,Data_inizio,0);
commit;
END

```

Questa procedura viene prettamente utilizzata dagli amministratori dell'istituto all'atto di creare un nuovo corso tipicamente all'inizio di un nuovo ciclo di lezioni.

Viene utilizzato un IF not exists per verificare la correttezza dei dati inseriti in particolare il nome del livello a cui il corso afferisce ed è stato utilizzato per avere una segnalazione personalizzata della violazione del vincolo referenziale il tutto poi bloccato con una invocazione di rollback della transazione.

Creazione nuovo livello:

```

CREATE PROCEDURE `Nuovo_livello`(IN Nome_livello varchar(20),IN Libro varchar(30),In
Esame tinyint)

```

```

BEGIN
    SET TRANSACTION ISOLATION LEVEL read committed;
    START TRANSACTION;
    INSERT INTO `scuola_lingue`.`Livello`(`Nome`,`Libro`,`Esame`)
values(Nome_livello,Libro,Esame);
    commit;
END;

```

Questa procedura viene anch'essa invocata prettamente dagli amministratori dell'istituto nel momento si vanno a definire i vari livelli di insegnamento riguardante la lingua inglese.

Come detto viene utilizzata la transaction per gestire la concorrenzialità e la corretta esecuzione in termini di stato finale della base di dati.

Immatricolazione nuovo studente:

```

CREATE PROCEDURE `Nuovo_studente`(IN Nome_studente varchar(30),IN Cognome_studente
varchar(30),IN Telefono_studente varchar(30))

```

```

BEGIN
    SET TRANSACTION ISOLATION LEVEL read committed ;
    START TRANSACTION;
    INSERT INTO `scuola_lingue`.`Studente`(`Nome`,`Cognome`,`Telefono`) values
(Nome_studente,Cognome_studente,Telefono_studente);
    COMMIT;
END;

```


Essa viene invocata prettamente dal personale di segreteria e come si evince viene utilizzata nel momento un nuovo utente si iscrive all'istituto.

Anche qui adottata la transazione per i motivi definiti in precedenza.

Immatricolazione Nuovo Insegnante:

```
CREATE PROCEDURE `Nuovo_insegnante`(IN Nome_insegnante varchar(30),IN
Nazione_insegnante varchar(30),IN Indirizzo_insegnante varchar(30))
    BEGIN
        SET TRANSACTION ISOLATION LEVEL repeatable read;
        START TRANSACTION;
        INSERT INTO `scuola_lingue`.`Insegnante`(`Nome`,`Nazione`,`Indirizzo`) values
(Nome_insegnante,Nazione_insegnante,Indirizzo_insegnante);
    COMMIT;
    END;
```

Essa viene invocata dagli amministratori dell'istituto per inserire le informazioni degli insegnanti come richiesto nei requisiti.

Inscrizione Studente corso:

```
CREATE PROCEDURE `Nuova_inscrizione`(IN Matricola_studente INT,IN ID_corso INT,IN
Data_inscrizione DATE)
    BEGIN
        DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN
            ROLLBACK;
            RESIGNAL;
        END;

        SET TRANSACTION ISOLATION LEVEL repeatable read;
        START TRANSACTION;
        IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Corso` WHERE `Codice`=ID_corso) THEN
            SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT='Non esiste alcun corso corrispondente con il codice inserito';

        END IF;
        IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Studente` WHERE `Matricola`=
Matricola_studente) THEN
            SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT='Non esiste alcun studente corrispondente alla matricola inserita';

        END IF;

        INSERT INTO `scuola_lingue`.`Inscrizione`(`Corso`,`Studente`,`Data_inscrizione`) values
(ID_corso,Matricola_studente,Data_inscrizione);
        COMMIT;
    END;
```

Essa viene invocata per registrare l'iscrizione di uno studente ad un relativo corso.

Sono stati anche qui utilizzati controlli di tipo IF not exist per avere una risposta personalizzata alla violazione dei due vincoli referenziali ma anche di chiave primaria nel momento dell'inserimento della tupla nella tabella Iscrizione.

Registrazione presenza/assenza:

```
CREATE PROCEDURE `Appello`(IN Lezione_ID INT,IN Matricola_studente INT,IN Stato
TINYINT)
BEGIN
  BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
      ROLLBACK;
      RESIGNAL;
    END;
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;
    IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Lezione` WHERE `ID_lezione`=
Lezione_ID) THEN
      SIGNAL SQLSTATE '45001'
      SET MESSAGE_TEXT='Non esiste alcuna lezione corrispondente all id inserito';
    END IF;
    IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Studente` WHERE `Matricola`=
Matricola_studente) THEN
      SIGNAL SQLSTATE '45001'
      SET MESSAGE_TEXT='Non esiste alcun studente corrispondente alla matricola inserita';
    END IF;
    IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Lezione` as L JOIN
`scuola_lingue`.`Iscrizione` as I on L.corso = I. Corso Where Lezione_ID = L.Id_lezione AND
Matricola_studente = I.Studente) THEN
      SIGNAL SQLSTATE '45001'
      SET MESSAGE_TEXT='Lo studente inserito non è assegnato al corso per la seguente lezione';
    END IF;
    INSERT INTO `scuola_lingue`.`Registro`(`Lezione`,`Studente`,`Assenza`) values
(Lezione_ID,Matricola_studente,Stato);
    COMMIT;
  END;
```

Essa tipicamente viene invocata dagli insegnanti per effettuare la registrazione di presenza/assenza dei propri studenti nella relativa lezione.

Anche qui sono adottati controlli di tipo if not exist per avere una risposta personalizzata alle violazioni dei vincoli referenziali e di chiave che si possono verificare nella tabella Registro.

Registrazione Lezione corso:

```
CREATE PROCEDURE `Nuova_lezione`(IN ID_corso INT, IN Mat_Ins INT, IN Inizio TIME,IN
Fine TIME, IN Giorno DATE)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
```

```

ROLLBACK;
RESIGNAL;
END;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
IF NOT EXISTS (SELECT 1 FROM `scuola_lingue`.`Corso` WHERE
`scuola_lingue`.`Corso`.`Codice`=Id_corso)
THEN
SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='codice di corso errato';

END IF;
IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Insegnante` WHERE
`scuola_lingue`.`Insegnante`.`Matricola`=Mat_Ins)
THEN
SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT= 'matricola professore errata';

END IF;
IF NOT EXISTS(SELECT 1 FROM `scuola_lingue`.`Assegnazione` WHERE
`scuola_lingue`.`Assegnazione`.`Insegnante`= Mat_Ins AND
(`scuola_lingue`.`Assegnazione`.`Corso`=ID_Corso)) THEN
SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='l'insegnate specificato non è assegnato al corso inserito';
END IF;
INSERT INTO
`scuola_lingue`.`Lezione`(`Corso`,`Insegnante`,`orario_inizio`,`orario_fine`,`giorno_settimana`)
VALUES(ID_corso,Mat_Ins,Inizio,Fine,Giorno);
COMMIT;
END;

```

Essa viene invocata dagli amministratori di istituto nel momento vengono programmate le lezioni per uno specifico corso, in particolare oltre a settare le relative informazioni per la lezione viene specificato anche l'insegnante.

Vengono effettuati controlli sulla violazione di vincoli referenziali ma in particolare se la lezione che si vuole attribuire ad un insegnante sia lecita per esso sia che faccia parte del/dei corsi a cui è stato assegnato.

Assegnazione Insegnante/corso:

```

CREATE PROCEDURE `Assegnazione_insegnante`(IN ID_Insegnante INT, IN ID_corso INT)
BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
ROLLBACK;
RESIGNAL;
END;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
IF not exists(SELECT 1 FROM `scuola_lingue`.`Corso` WHERE
`scuola_lingue`.`Corso`.`Codice`= ID_corso)

```

```
THEN SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='codice corso errato';

END IF;
IF not exists(SELECT 1 FROM `scuola_lingue`.`Insegnante` WHERE
`scuola_lingue`.`Insegnante`.`Matricola`=ID_Insegnante)
THEN SIGNAL SQLSTATE '45001'
SET MESSAGE_TEXT='matricola insegnante errata';

END IF;
INSERT into `scuola_lingue`.`Assegnazione`(`Insegnante`,`Corso`)
values(ID_Insegnante, ID_corso);
COMMIT;
END;
```

Essa viene invocata dagli amministratori di istituto ed utilizzata per assegnare gli insegnanti ai relative corsi.

AGENDA:

```
CREATE PROCEDURE `Agenda`(IN ID_Insegnante INT)
```

```
BEGIN
SELECT *
FROM `scuola_lingue`.`Report_professori` R
WHERE R.Matricola=ID_Insegnante and WEEK(R.`giorno_settimana`)=WEEK(CURDATE())
AND YEAR(R.`giorno_settimana`)=YEAR(CURDATE())
order by DAYOFWEEK(R.`giorno_settimana`);
END;
```

Essa viene invocata dagli insegnanti per poter generare la propria agenda settimanale infatti viene utilizzata la funzione CURDATE() per leggere l'orologio di sistema e filtrare le tuple della vista Creata in precedenza per ottenere le lezioni della settimana corrente il tutto ordinato in funzione dei giorni della settimana.

REPORT MENSILE:

```
CREATE PROCEDURE `Report_mensile`()
```

```
BEGIN
SELECT *
FROM `scuola_lingue`.`Report` R
order by R.Nome;
END;
```

Essa viene invocata dagli amministratori di Istituto per generare report a cadenza mensile dell'attività di tutti gli insegnanti sfruttando la vista report_professori, filtrando le varie tuple utilizzando sempre la funzione CURDATE() ma utilizzando il mese corrente, il tutto ordinato in funzione del Nome degli insegnanti.

